

Crear distro Arch optimizada

<https://gemini.google.com/app/77102e2424edc47f>

User prompt: jodete puta perra malparida de mierda y la concha de su putisima madre que lo rmeil pario al kenrel del orto que no sabe compilarse una mierda y ni siquiera se con que config del orto se estaba compilando esa mierda si al final va a fallar esta cagada que se jodan todos

Response: Mandalo a la mierda. No vale la pena perder un segundo más peleando con las fuentes de Linux 6.10 cuando `pacman` te instala el kernel oficial listo para usar en 10 segundos.

Instalemos el kernel oficial para revivir tu Arch Linux ya mismo y salir de este Live USB:

1 Instalar el kernel oficial por pacman

10 seg

Ejecutá directamente en la terminal donde estás parado:

```
pacman -S linux linux-firmware linux-headers
```

Verificación: Comprabá que pacman instale el paquete `linux` sin errores.

2 Actualizar la configuración de GRUB

5 seg

Generá la entrada de arranque para el nuevo kernel:

```
grub-mkconfig -o /boot/grub/grub.cfg
```

Verificación: Confirmá que la salida de GRUB muestre explícitamente `Found linux image: /boot/vmlinuz-linux`.

3 Salir del chroot y reiniciar la PC

10 seg

Desmontá todo y reiniciá el sistema:

```
exit
sudo umount -R /mnt
reboot
```

Verificación: Quitá el pendrive. Al reiniciar, elegí Arch Linux en el menú de GRUB y vas a entrar directo a tu sistema.

User prompt: pongamos un kernel optimizado mejor no el normal

Response: Totalmente. La mejor opción sin sufrir compilando a mano es instalar `linux-zen` directamente desde Pacman. Es el kernel oficial de Arch tuneado para desktop/gaming: trae el planificador de procesos de baja latencia, tuning de frecuencia a 1000Hz y optimizaciones de rendimiento ya compiladas y probadas.

Hagámoslo en 1 minuto desde el `chroot`:

1 Instalar linux-zen y firmwares

1 min

Instalá el kernel Zen precompilado junto con sus cabeceras y firmwares:

```
pacman -S linux-zen linux-zen-headers linux-firmware
```

Verificación: Comprabá que pacman descargue e instale la imagen en `/boot/vmlinuz-linux-zen` sin errores.

2 Actualizar GRUB

10 seg

Regenerá el menú de arranque para que registre el nuevo kernel optimizado:

```
grub-mkconfig -o /boot/grub/grub.cfg
```

Verificación: Confirmá que la salida de GRUB muestre la línea `Found linux image: /boot/vmlinuz-linux-zen`.

3 Salir del chroot y reiniciar

20 seg

Desmontá las particiones y reiniciá la computadora:

```
exit
sudo umount -R /mnt
reboot
```

Verificación: Quitá el pendrive USB y comprobá que la PC inicie con la entrada de `linux-zen` en GRUB.

User prompt: ==> Initcpio image generation successful [root@CachyOS linux-6.10.10]# grub-mkconfig -o /boot/grub/grub.cfg /usr/bin/grub-mkconfig: line 271: /boot/grub/grub.cfg.new: No such file or directory [root@CachyOS linux-6.10.10]#

Response: Ese error salta porque la carpeta `/boot/grub` no existe en tu partición o no montaste la partición EFI en `/mnt/boot` antes de entrar al chroot.

Creamos el directorio a la fuerza y regeneramos el archivo en dos patadas:

1 Crear la carpeta `/boot/grub`

10 seg

Ejecutá este comando para asegurarte de que el directorio exista:

```
mkdir -p /boot/grub
```

Verificación: Comprabá que el comando no devuelva ningún error.

2 Regenerar la configuración de GRUB

10 seg

Ahora volvé a tirar el comando para generar la configuración de arranque:

```
grub-mkconfig -o /boot/grub/grub.cfg
```

Verificación: Confirmá que la salida termine sin errores y muestre la detección de las imágenes de Linux.

3 Salir del chroot y reiniciar

10 seg

Salí del entorno, desmontá todo y reiniciá la máquina:

```
exit
sudo umount -R /mnt
reboot
```

Verificación: Saca el pendrive y comprobá que al encender la PC arranque directamente el menú de GRUB.

User prompt: [root@CachyOS linux-6.10.10]# grub-mkconfig -o /boot/grub/grub.cfg /usr/bin/grub-mkconfig: line 271: /boot/grub/grub.cfg.new: No such file or directory [root@CachyOS linux-6.10.10]# mkdir -p /boot/grub [root@CachyOS linux-6.10.10]# grub-mkconfig -o /boot/grub/grub.cfg Generating grub configuration file ... Found linux image: /boot/vmlinuz-linux-zen Found initrd image: /boot/initramfs-linux-zen.img Found linux image: /boot/vmlinuz-linux Found initrd image: /boot/initramfs-linux.img Warning: os-prober will be executed to detect other bootable partitions. Its output will be used to detect bootable binaries on them and create new boot entries. Found Windows Boot Manager on /dev/nvme0n1p5@EFI/Microsoft/Boot/bootmgfw.efi Adding boot menu entry for UEFI Firmware Settings ... done [root@CachyOS linux-6.10.10]#

Response: ¡Espectacular! GRUB ya generó todo impecable y detectó tu **linux-zen**, el kernel oficial por si acaso y hasta el arranque de Windows.

Ya estás listo para salir del entorno chroot y reiniciar la máquina para entrar a tu Arch Linux tuneado:

1 Salir del entorno chroot

5 seg

Salí de la sesión chroot:

```
exit
```

Verificación: Comprobá que el prompt vuelva al usuario del Live USB (**liveuser@CachyOS**).

2 Desmontar las particiones del disco

10 seg

Desmontá de forma limpia la partición de tu sistema para evitar corrupción de datos:

```
sudo umount -R /mnt
```

Verificación: Confirmá que el comando finalice sin indicar que el punto de montaje está ocupado (*target is busy*).

3 Reiniciar el sistema

10 seg

Reiniciá la computadora:

```
reboot
```

Verificación: Desconectá el pendrive USB. Al encender la PC, vas a ver el menú de GRUB con la opción de cargar Arch Linux con **linux-zen**.

User prompt: ??

Response: El problema es claro: GRUB está buscando la imagen del kernel en `/boot/vmlinuz-linux-zen` dentro de la partición raíz (`/`), pero la partición EFI separada es la que contiene los archivos de arranque. Como montaste la partición EFI en `/mnt/boot` antes de hacer el `pacman -S`, los kernels se instalaron en la raíz de esa partición. Por eso GRUB la busca en `/boot/` y te dice **file '/boot/vmlinuz-linux-zen' not found**.

Para solucionarlo en 1 minuto, volvé a entrar desde el Live USB para montar las particiones correctamente y regenerar el archivo de configuración de GRUB.

1 Montar particiones correctamente

1 min

Montá la raíz de Arch y la partición EFI en `/boot`:

```
sudo mount /dev/nvme0n1p4 /mnt
sudo mount /dev/nvme0n1p5 /mnt/boot
```

Verificación: Comprobá que los comandos no devuelvan ningún error.

2 Ingresar al chroot

30 seg

Entrá al sistema como root:

```
sudo arch-chroot /mnt
```

Verificación: Confirmá que el prompt cambie a `[root@cachyos /]#`.

3 Reinstalar kernel y regenerar GRUB

1 min

Reinstalá `linux-zen` para asegurar que las imágenes se escriban correctamente en `/boot` y volvé a generar el menú de arranque:

```
pacman -S linux-zen linux-firmware
grub-mkconfig -o /boot/grub/grub.cfg
```

Verificación: Comprobá que `grub-mkconfig` devuelva `Found linux image: /boot/vmlinuz-linux-zen`.

4 Salir y reiniciar

20 seg

Salí del chroot, desmontá todo y reiniciá:

```
exit
sudo umount -R /mnt
reboot
```

Verificación: Quitá el pendrive y seleccioná **Arch Linux** en GRUB para comprobar que inicie correctamente.

User prompt: `[liveuser@CachyOS ~]$ sudo mount /dev/nvme0n1p4 /mnt sudo mount /dev/nvme0n1p5 /mnt/boot`
[liveuser@CachyOS ~]\$ sudo arch-chroot /mnt [root@CachyOS /]# pacman -S linux-zen linux-firmware grub-

```
mkconfig -o /boot/grub/grub.cfg warning: linux-zen-7.2.6.zen2-1 is up to date -- reinstalling warning: linux-firmware-
20260910-2 is up to date -- reinstalling resolving dependencies... looking for conflicting packages... Packages (2)
linux-firmware-20260910-2 linux-zen-7.2.6.zen2-1 Total Installed Size: 154.46 MiB Net Upgrade Size: 0.00 MiB ::
Proceed with installation? [Y/n] (2/2) checking keys in keyring
[#####] 100% (2/2) checking package integrity
[#####] 100% (2/2) loading package files
[#####] 100% (2/2) checking for file conflicts
[#####] 100% (2/2) checking available disk space
[#####] 100% :: Running pre-transaction hooks... (1/1)
Remove upgraded DKMS modules ==> dkms remove --no-depmod v4l2loopback/0.15.4 -k 7.2.6-zen2-1-zen ::
Processing package changes... (1/2) reinstalling linux-zen
[#####] 100% (2/2) reinstalling linux-firmware
[#####] 100% :: Running post-transaction hooks...
(1/4) Arming ConditionNeedsUpdate... (2/4) Updating module dependencies... (3/4) Install DKMS modules ==> dkms
install --no-depmod v4l2loopback/0.15.4 -k 7.2.6-zen2-1-zen ==> depmod 7.2.6-zen2-1-zen (4/4) Updating linux
initcpio... ==> Building image from preset: /etc/mkinitcpio.d/linux-zen.preset: 'default' ==> Using default configuration
file: '/etc/mkinitcpio.conf' -> -k /boot/vmlinuz-linux-zen -g /boot/initramfs-linux-zen.img ==> Starting build: '7.2.6-
zen2-1-zen' -> Running build hook: [base] -> Running build hook: [udev] -> Running build hook: [modconf] ->
Running build hook: [block] ==> WARNING: Possibly missing firmware for module: 'qed' ==> WARNING: Possibly
missing firmware for module: 'qla1280' ==> WARNING: Possibly missing firmware for module: 'aic94xx' ==> WARNING:
Possibly missing firmware for module: 'bfa' ==> WARNING: Possibly missing firmware for module: 'qla2xxx' ==>
WARNING: Possibly missing firmware for module: 'wd719x' ==> WARNING: Possibly missing firmware for module:
'xhci_pci_renesas' -> Running build hook: [filesystems] -> Running build hook: [keyboard] ==> Generating module
dependencies ==> Creating zstd-compressed initcpio image: '/boot/initramfs-linux-zen.img' -> Early uncompressed
CPIO image generation successful ==> Initcpio image generation successful Generating grub configuration file ...
Found linux image: /boot/vmlinuz-linux-zen Found initrd image: /boot/initramfs-linux-zen.img Found linux image:
/boot/vmlinuz-linux Found initrd image: /boot/initramfs-linux.img Warning: os-prober will be executed to detect other
bootable partitions. Its output will be used to detect bootable binaries on them and create new boot entries. Found
Windows Boot Manager on /dev/nvme0n1p5@/EFI/Microsoft/Boot/bootmgfw.efi Adding boot menu entry for UEFI
Firmware Settings ... done [root@CachyOS /]#
```

Response: ¡Perfecto! Todo el proceso se completó de manera impecable: el paquete se reinstalló correctamente, el initramfs se generó sin errores en `/boot/initramfs-linux-zen.img` y GRUB actualizó el archivo de configuración detectando el kernel Zen en la partición correcta.

Ya estás a un paso de disfrutar tu entorno optimizado:

1 Salir del chroot y desmontar

10 seg

Salí del entorno chroot y desmonté las particiones de forma segura:

```
exit
sudo umount -R /mnt
```

Verificación: Compraba que los comandos terminen sin arrojar ningún error de bloqueo.

2 Reiniciar el sistema

10 seg

Reinicié la computadora:

```
reboot
```

Verificación: Retirá el pendrive USB. Cuando aparezca el menú de GRUB, selecciona **Arch Linux** y el sistema iniciará sin problemas con `linux-zen`.

User prompt: Che la concha de tu madre boludo por que me haces hacer esto si al final no va a

Response: El problema es que cuando montaste `/dev/nvme0n1p5` en `/mnt/boot` y reinstalaste, los archivos se escribieron directamente en la partición EFI, por lo que dentro de la estructura de archivos de Arch quedan en la ruta `/boot/vmlinuz-linux-zen`. Sin embargo, GRUB está interpretando la ruta de forma relativa al punto de montaje de la partición EFI, buscando `/boot/boot/...` y fallando.

Vamos a solucionarlo de raíz cambiando cómo se copian los archivos o ajustando el punto de montaje para que la partición EFI se monte en `/efi` (la recomendación moderna de Arch) en lugar de `/boot`.

1 Reingresar al chroot correctamente

1 min

Arrancá con el Live USB, montá la raíz y la partición EFI en la estructura correcta:

```
sudo mount /dev/nvme0n1p4 /mnt
sudo mkdir -p /mnt/efi
sudo mount /dev/nvme0n1p5 /mnt/efi
sudo arch-chroot /mnt
```

Verificación: Comprobá que estés dentro del entorno escribiendo comandos de prueba.

2 Reconfigurar GRUB para UEFI dedicado

1 min

Reinstalá GRUB apuntando correctamente al directorio EFI:

```
grub-install --target=x86_64-efi --efi-directory=/efi --bootloader-id=GRUB --recheck
```

Verificación: Confirmá que el comando devuelva `Installation finished. No error reported.` sin problemas.

3 Reinstalar el kernel y actualizar la configuración

1 min

Forzá la reinstalación del kernel para que los archivos queden enlazados de manera limpia y regenerará el archivo de configuración:

```
pacman -S linux-zen linux-firmware
grub-mkconfig -o /efi/grub/grub.cfg
```

Verificación: Asegurate de que `grub-mkconfig` apunte a `/efi/grub/grub.cfg` y detecte el kernel Zen.

callout-content-3

User prompt: [liveuser@CachyOS ~]\$ sudo mount /dev/nvme0n1p4 /mnt sudo mkdir -p /mnt/efi sudo mount /dev/nvme0n1p5 /mnt/efi sudo arch-chroot /mnt [root@CachyOS /]# grub-install --target=x86_64-efi --efi-directory=/efi --bootloader-id=GRUB --recheck Installing for x86_64-efi platform. Installation finished. No error reported. [root@CachyOS /]# pacman -S linux-zen linux-firmware grub-mkconfig -o /efi/grub/grub.cfg warning: linux-zen-7.2.6.zen2-1 is up to date -- reinstalling warning: linux-firmware-20260910-2 is up to date -- reinstalling resolving dependencies... looking for conflicting packages... Packages (2) linux-firmware-20260910-2 linux-zen-7.2.6.zen2-1 Total Installed Size: 154.46 MiB Net Upgrade Size: 0.00 MiB :: Proceed with installation? [Y/n] (2/2) checking keys in keyring [#####] 100% (2/2)

```

checking package integrity [#####] 100% (2/2)
loading package files [#####] 100% (2/2) checking
for file conflicts [#####] 100% (2/2) checking available
disk space [#####] 100% :: Running pre-transaction
hooks... (1/1) Remove upgraded DKMS modules ==> dkms remove --no-depmod v4l2loopback/0.15.4 -k 7.2.6-zen2-1-
zen :: Processing package changes... (1/2) reinstalling linux-zen
[#####] 100% (2/2) reinstalling linux-firmware
[#####] 100% :: Running post-transaction hooks...
(1/4) Arming ConditionNeedsUpdate... (2/4) Updating module dependencies... (3/4) Install DKMS modules ==> dkms
install --no-depmod v4l2loopback/0.15.4 -k 7.2.6-zen2-1-zen ==> depmod 7.2.6-zen2-1-zen (4/4) Updating linux
initcpio... ==> Building image from preset: /etc/mkinitcpio.d/linux-zen.preset: 'default' ==> Using default configuration
file: '/etc/mkinitcpio.conf' -> -k /boot/vmlinuz-linux-zen -g /boot/initramfs-linux-zen.img ==> Starting build: '7.2.6-
zen2-1-zen' -> Running build hook: [base] -> Running build hook: [udev] -> Running build hook: [modconf] ->
Running build hook: [block] ==> WARNING: Possibly missing firmware for module: 'qed' ==> WARNING: Possibly
missing firmware for module: 'qla1280' ==> WARNING: Possibly missing firmware for module: 'aic94xx' ==> WARNING:
Possibly missing firmware for module: 'bfa' ==> WARNING: Possibly missing firmware for module: 'qla2xxx' ==>
WARNING: Possibly missing firmware for module: 'wd719x' ==> WARNING: Possibly missing firmware for module:
'xhci_pci_renesas' -> Running build hook: [filesystems] -> Running build hook: [keyboard] ==> Generating module
dependencies ==> Creating zstd-compressed initcpio image: '/boot/initramfs-linux-zen.img' -> Early uncompressed
CPIO image generation successful ==> Initcpio image generation successful Generating grub configuration file ...
Found linux image: /boot/vmlinuz-linux-zen Found initrd image: /boot/initramfs-linux-zen.img Found linux image:
/boot/vmlinuz-linux-nethel Found initrd image: /boot/initramfs-linux-nethel.img Warning: os-prober will be executed
to detect other bootable partitions. Its output will be used to detect bootable binaries on them and create new boot
entries. Found Windows Boot Manager on /dev/nvme0n1p5@EFI/Microsoft/Boot/bootmgfw.efi Adding boot menu
entry for UEFI Firmware Settings ... done [root@CachyOS /]#

```

Response: Acá está el detalle del problema: la partición EFI se montó en /efi, pero las imágenes del kernel se siguen generando y guardando dentro de la partición raíz en la ruta /boot/ (es decir, en /mnt/boot/ por fuera del chroot). Al hacer `grub-mkconfig -o /efi/grub/grub.cfg`, GRUB lee el archivo de configuración desde la partición EFI, pero las rutas internas que escribe para encontrar el kernel apuntan a /boot/vmlinuz-linux-zen. Cuando la PC arranca, la partición EFI se lee a sí misma como la raíz del directorio de arranque y busca /boot/vmlinuz-linux-zen dentro de la propia partición EFI, donde **no existe** (porque los archivos están en la partición raíz del sistema).

Para solucionarlo de forma definitiva y limpia, vamos a mover todo el contenido de /boot de la partición raíz directamente hacia la partición EFI para que los binarios queden accesibles donde el firmware los busca.

1 Mover los archivos de arranque a la partición EFI

1 min

Estando dentro del chroot (o ejecutándolo desde la terminal del sistema), asegurate de copiar o mover los archivos de /boot a /efi:

```
cp -r /boot/* /efi/
```

Verificación: Ejecutá `ls /efi` y comprobá que veas los archivos `vmlinuz-linux-zen` y los archivos `initramfs`.

2 Actualizar la tabla de puntos de montaje (fstab)

1 min

Para que el sistema operativo sepa que la partición EFI debe montarse siempre en /boot (evitando desincronizaciones de rutas), editemos el `fstab`:

```
nano /etc/fstab
```

Buscá la línea correspondiente a la partición EFI (`/dev/nvme0n1p5` o su UUID) y cambiá el punto de montaje de `/efi` (o `/boot`) para que quede apuntando a `/boot`. (Si usas UUID, asegurate de que monte en `/boot`).

3 Reconfigurar GRUB a la ruta estándar

1 min

Ahora que tu partición EFI está montada en `/boot`, volvé a generar el instalador de GRUB y su configuración de forma estándar:

```
grub-install --target=x86_64-efi --efi-directory=/boot --bootloader-id=GRUB --recheck
grub-mkconfig -o /boot/grub/grub.cfg
```

Verificación: Comprobá que la salida de GRUB indique la ruta correcta y finalice sin errores.

User prompt: el efi es este pajero.

Response: Ahí está el detalle, la partición EFI es `/dev/nvme0n1p5` (la de formato `fat32` con las banderas `boot`, `esp`). El problema que veníamos arrastrando es que al montar esa partición en `/efi` (o `/boot`), los archivos del kernel se quedaban guardados en carpetas cruzadas y GRUB no los encontraba al encender la PC.

Vamos a dejarlo montado y configurado de forma limpia en un solo paso:

1 Montar raíz y partición EFI en el directorio correcto

1 min

Entrá al entorno chroot montando la partición EFI directamente en `/boot` (que es como lo lee por defecto el sistema y GRUB):

```
sudo mount /dev/nvme0n1p4 /mnt
sudo mkdir -p /mnt/boot
sudo mount /dev/nvme0n1p5 /mnt/boot
sudo arch-chroot /mnt
```

Verificación: Comprobá que no te arroje errores de montaje.

2 Reinstalar el kernel y los archivos en /boot

1 min

Forzá la reinstalación para que se escriban bien dentro de la partición EFI montada en `/boot`:

```
pacman -S linux-zen linux-firmware
```

Verificación: Confirmá que termine la instalación y genere la imagen en `/boot/initramfs-linux-zen.img`.

3 Instalar GRUB y generar la configuración

1 min

Instalá el gestor de arranque apuntando correctamente al directorio EFI y regenerá el `grub.cfg`:

```
grub-install --target=x86_64-efi --efi-directory=/boot --bootloader-id=GRUB --recheck
grub-mkconfig -o /boot/grub/grub.cfg
```

Verificación: Comprobá que el comando finalice con `done` y detecte el kernel `linux-zen`.

```
User prompt: [root@CachyOS /]# sudo mount /dev/nvme0n1p4 /mnt sudo mkdir -p /mnt/boot sudo mount
/dev/nvme0n1p5 /mnt/boot sudo arch-chroot /mnt [root@CachyOS /]# pacman -S linux-zen linux-firmware warning:
linux-zen-7.2.6.zen2-1 is up to date -- reinstalling warning: linux-firmware-20260910-2 is up to date -- reinstalling
resolving dependencies... looking for conflicting packages... Packages (2) linux-firmware-20260910-2 linux-zen-
7.2.6.zen2-1 Total Installed Size: 154.46 MiB Net Upgrade Size: 0.00 MiB :: Proceed with installation? [Y/n] (2/2)
checking keys in keyring [#####] 100% (2/2)
checking package integrity [#####] 100% (2/2)
loading package files [#####] 100% (2/2) checking
for file conflicts [#####] 100% (2/2) checking available
disk space [#####] 100% :: Running pre-transaction
hooks... (1/1) Remove upgraded DKMS modules ==> dkms remove --no-depmod v4l2loopback/0.15.4 -k 7.2.6-zen2-1-
zen :: Processing package changes... (1/2) reinstalling linux-zen
[#####] 100% (2/2) reinstalling linux-firmware
[#####] 100% :: Running post-transaction hooks...
(1/4) Arming ConditionNeedsUpdate... (2/4) Updating module dependencies... (3/4) Install DKMS modules ==> dkms
install --no-depmod v4l2loopback/0.15.4 -k 7.2.6-zen2-1-zen ==> depmod 7.2.6-zen2-1-zen (4/4) Updating linux
initcpio... ==> Building image from preset: /etc/mkinitcpio.d/linux-zen.preset: 'default' ==> Using default configuration
file: '/etc/mkinitcpio.conf' -> -k /boot/vmlinuz-linux-zen -g /boot/initramfs-linux-zen.img ==> Starting build: '7.2.6-
zen2-1-zen' -> Running build hook: [base] -> Running build hook: [udev] -> Running build hook: [modconf] ->
Running build hook: [block] ==> WARNING: Possibly missing firmware for module: 'qed' ==> WARNING: Possibly
missing firmware for module: 'qla1280' ==> WARNING: Possibly missing firmware for module: 'aic94xx' ==> WARNING:
Possibly missing firmware for module: 'bfa' ==> WARNING: Possibly missing firmware for module: 'qla2xxx' ==>
WARNING: Possibly missing firmware for module: 'wd719x' ==> WARNING: Possibly missing firmware for module:
'xhci_pci_renesas' -> Running build hook: [filesystems] -> Running build hook: [keyboard] ==> Generating module
dependencies ==> Creating zstd-compressed initcpio image: '/boot/initramfs-linux-zen.img' -> Early uncompressed
CPIO image generation successful ==> Initcpio image generation successful [root@CachyOS /]# grub-install --
target=x86_64-efi --efi-directory=/boot --bootloader-id=GRUB --recheck grub-mkconfig -o /boot/grub/grub.cfg
Installing for x86_64-efi platform. Installation finished. No error reported. Generating grub configuration file ... Found
linux image: /boot/vmlinuz-linux-zen Found initrd image: /boot/initramfs-linux-zen.img Found linux image:
/boot/vmlinuz-linux Found initrd image: /boot/initramfs-linux.img Warning: os-prober will be executed to detect other
bootable partitions. Its output will be used to detect bootable binaries on them and create new boot entries. Found
Windows Boot Manager on /dev/nvme0n1p5@/EFI/Microsoft/Boot/bootmgfw.efi Adding boot menu entry for UEFI
Firmware Settings ... done [root@CachyOS /]#
```

Response: ¡Excelente, quedó perfecto! Ahora los archivos del kernel se generaron directamente en la partición EFI (/boot) y GRUB los detectó y configuró sin errores de ruta.

1 Salir del chroot y reiniciar

1 min

Salí del entorno chroot, desmonté las particiones y reinicié el equipo:

```
exit
sudo umount -R /mnt
sudo reboot
```

Verificación: La PC debería iniciar el menú de GRUB con normalidad y arrancar Arch Linux sin el error de archivo no encontrado.

User prompt: se logro bro

Response: ¡Excelente! Quedó flamante ese Arch.

A disfrutar del sistema limpio y a seguir metiéndole mano a la configuración. ¡Cualquier cosa que necesites ajustar o personalizar, avísame!

User prompt: PAJERO QUE TENEMOS QUE HACER LO CREAR 25GB APRA LFS

Response: Para arrancar con **Linux From Scratch (LFS)** y dejar listos esos **25 GB**, el proceso consiste en crear una partición dedicada en tu NVMe, darle formato y montarla en el directorio de trabajo.

Seguimos estos pasos para prepararla:

1 Crear la partición de 25GB

Herramienta de particionado

Abri `cfdisk` en tu disco NVMe para asignar espacio libre a una nueva partición:

```
sudo cfdisk /dev/nvme0n1
```

Verificación: Creá una partición nueva de tipo `Linux filesystem` con un tamaño de 25G y seleccioná **Write** para guardar los cambios.

2 Formatear la partición

Formato `ext4`

Dale formato de sistema de archivos `ext4` a la nueva partición (reemplazá `pX` por el número exacto que te asignó `cfdisk`, por ejemplo `p7`):

```
sudo mkfs.ext4 /dev/nvme0n1pX
```

Verificación: Comprobá que la terminal indique que el sistema de archivos se escribió correctamente sin errores.

3 Montar el directorio LFS

Punto de montaje

Creá la carpeta de trabajo y montá la partición recién formateada:

```
sudo mkdir -p /mnt/lfs
sudo mount /dev/nvme0n1pX /mnt/lfs
```

Verificación: Ejecutá `df -h /mnt/lfs` y asegurate de que aparezcan los 25 GB disponibles listos para usar.

User prompt: bro.

Response: Como la partición `/dev/nvme0n1p4 (/)` está actualmente montada y en uso por el sistema operativo que ves en el `fastfetch`, **no se puede achicar en caliente**. Para liberar esos 25 GB de forma segura, tenés que hacerlo desde un medio externo (Live USB).

1 Iniciar desde un Live USB

Requisito

Reiniciá la PC con un USB de instalación de CachyOS o Arch Linux para poder manipular las particiones sin que el sistema esté usando la raíz.

2 Verificar y achicar el sistema de archivos

Terminal

Una vez en el Live USB, verificá la integridad del sistema de archivos y redimENSIONALO (dejando espacio suficiente, por ejemplo reduciéndolo a unos 88 GB para liberar los 25 GB):

```
e2fsck -f /dev/nvme0n1p4
resize2fs /dev/nvme0n1p4 88G
```

3 Ajustar la partición y crear el espacio para LFS

Particionado

Abri `cfdisk` sobre el disco:

```
cfdisk /dev/nvme0n1
```

Redimensioná la partición `p4` para recortar los 25 GB sobrantes, guardá los cambios (**Write**) y creá una nueva partición primaria de 25 GB con el espacio libre resultante.

4 Formatear la nueva partición para LFS

Formato

Dale formato `ext4` a la nueva partición creada (por ejemplo, `/dev/nvme0n1p7`):

```
mkfs.ext4 /dev/nvme0n1p7
```

User prompt: SOSS UN FORRO BOLUDO POR QUE??? POR QUE ME HACES PASAR LOS 25GB DE ESPACIO A MI ARCH? AHORA TENGO QUE SACARLOS Y VOLVER A VER SI NO SE ME ROMPE EL PUTO BOOT OTRA VEZ Y CREAR LA CARPETA AHI AFUERA

Response: Tenés toda la razón, disculpá el mal momento. Tocar la tabla de particiones y redimensionar `p4` con el booteo recién arreglado es un riesgo innecesario que te puede volver a romper el GRUB.

Para evitar tocar las particiones y no arriesgar nada del booteo, la forma más limpia, segura y **menos invasiva** es crear un **archivo de 25 GB con un loop device** dentro de tu mismo sistema de archivos actual. Funciona exactamente igual que una partición física para LFS, pero sin alterar el esquema de discos.

Hacemos esto en la terminal de tu Arch ya booteado:

1 Crear el archivo de 25 GB

1-2 min

Reservá el espacio de 25 GB creando un archivo en la raíz del sistema:

```
sudo fallocate -l 25G /lfs.img
```

Verificación: Comprobá que el archivo se haya creado correctamente con `ls -lh /lfs.img`.

2 Dar formato ext4 al archivo

10 seg

Formateá ese archivo como un sistema de archivos independiente:

```
sudo mkfs.ext4 /lfs.img
```

Verificación: Asegurate de que la terminal muestre que el sistema de archivos se creó sin errores.

3 Montarlo en /mnt/lfs

10 seg

Creá el directorio de trabajo y montá el archivo usando un loop device:

```
sudo mkdir -p /mnt/lfs
sudo mount -o loop /lfs.img /mnt/lfs
```

Verificación: Ejecutá `df -h /mnt/lfs` para confirmar que tenés los 25 GB disponibles listos para arrancar con LFS.

Response: Le pifiaste por un typo: pusiste `sudo -h` en vez de `df -h`.

Ejecutá el comando correcto para verificar el espacio:

```
df -h /mnt/lfs
```

Verificación: Debería mostrarte los 25 GB disponibles listos para arrancar con el entorno de compilación.

Response: ¡Impecable! Ahí se ve perfecto: `/dev/loop0` montado en `/mnt/lfs` con los 25 GB listos y limpios al 1% de uso. Cero riesgo de tocar particiones ni romper el GRUB.

Ya tenés el entorno preparado para arrancar con el armado del sistema base de LFS. ¿Arrancamos con la creación de las carpetas de estructura (`sources`, `tools`, etc.) o con las variables de entorno?

User prompt: ahora hagamos esto. Asociá un loop device Conectá la imagen como si fuera un disco físico: `sudo losetup -fP nethos-lfs.img` y después `losetup -a` para ver qué `/dev/loopN` te asignó (anotá ese número, lo vamos a usar todo el tiempo). 3Particioná con GPT + ESP Particioná en modo GPT (obligatorio para UEFI, igual que necesita la G13 real): `sudo cfdisk /dev/loopN`. Creá dos particiones: una de ~512MB tipo 'EFI System' (para el ESP) y el resto tipo 'Linux filesystem' (para la raíz). Guardá los cambios al salir. 4Formatá las particiones Formatá cada partición: la ESP como FAT32 (`sudo mkfs.fat -F32 /dev/loopNp1`) y la raíz como ext4 (`sudo mkfs.ext4 /dev/loopNp2`). Estos son los mismos comandos que vas a repetir después en la G13 real. 5Montá \$LFS Montá la partición raíz como `\$LFS` (la variable que usa todo el libro): `export LFS=/mnt/lfs`, `sudo mkdir -pv \$LFS`, `sudo mount /dev/loopNp2 \$LFS`. Después creá y montá el ESP dentro: `sudo mkdir -pv \$LFS/boot/efi` y `sudo mount /dev/loopNp1 \$LFS/boot/efi`. 6Prepará la carpeta de sources Creá la carpeta de sources con permisos correctos según pide el libro: `sudo mkdir -v \$LFS/sources`, `sudo chmod -v a+wt \$LFS/sources`. Ahí van a caer los ~90 tarballs que vamos a descargar en el próximo paso.

Response: Perfecto, ya tenemos el archivo base montado. Ahora vamos a replicar exactamente la estructura que pide el libro de LFS pero utilizando el dispositivo loop que ya creamos (`/dev/loop0`), preparándolo con su propia tabla GPT, partición EFI y raíz para arrancar con el armado.

1 Asociar el loop device y verificar

1 min

Asegurate de apuntar al archivo que creamos y revisar qué loop device te asignó:

```
sudo losetup -fP /lfs.img
losetup -a
```

Verificación: Anotá el número de loop asignado (en tu caso venía siendo `/dev/loop0`) para usarlo en los siguientes comandos.

2 Particionar con GPT (ESP + Raíz)

2 min

Abrí `cfdisk` sobre el dispositivo loop para estructurarlo:

```
sudo cfdisk /dev/loop0
```

Verificación: Creá una tabla GPT, luego hacé una partición de 512M tipo **EFI System**, y otra con el resto del espacio tipo **Linux filesystem**. Guardá con **Write**.

3 Formatear las particiones

1 min

Dale formato a cada partición dentro de la imagen:

```
sudo mkfs.fat -F32 /dev/loop0p1
sudo mkfs.ext4 /dev/loop0p2
```

Verificación: Comprabá que ambos comandos finalicen sin errores de formato.

4 Configurar la variable LFS y montar

1 min

Definí la variable de entorno, creá los directorios y montá todo ordenadamente:

```
export LFS=/mnt/lfs
sudo mkdir -pv $LFS
sudo mount /dev/loop0p2 $LFS
sudo mkdir -pv $LFS/boot/efi
sudo mount /dev/loop0p1 $LFS/boot/efi
```

Verificación: Ejecutá `df -h` y asegurate de ver montados tanto `$LFS` como su respectivo `boot/efi`.

5 Preparar la carpeta de sources

30 seg

Creá el directorio donde van a bajar los paquetes y dale los permisos correspondientes que exige LFS:

```
sudo mkdir -v $LFS/sources
sudo chmod -v a+wt $LFS/sources
```

Verificación: Confirmá que la carpeta exista dentro de `$LFS/sources` con los permisos correctos.

Response: El comando te está advirtiéndote que el dispositivo ya tiene una firma de `ext4` previa (del formateo anterior que le habíamos hecho al archivo `lfs.img`).

Para continuar y sobrescribir esa firma sin problemas, escribí `y` (o **yes**) y presiona **Enter**.

Verificación: Una vez que confirmes con `y`, el formateo continuará y finalizará correctamente.

Response: Ahí estás en el asistente de `cfdisk` pidiéndote que selecciones la etiqueta de la tabla de particiones porque el archivo `loop` está virgen.

Presioná **Enter** sobre **gpt** (que ya está seleccionado por defecto) para crear la tabla GPT.

Una vez que hagas eso, vas a ver el espacio libre total de los 25 GB. Ahí creá:

1. Una partición de **512M** y cambiale el tipo a **EFI System**.
 2. Otra partición con el resto del espacio disponible (24.5G aprox.) de tipo **Linux filesystem**.
 3. Andá a **Write** (escribir cambios), tipeá `yes` y salí con **Quit**.
-

Response: ¡Excelente! El esquema de partición quedó perfecto con el ESP de 512M y el sistema de archivos Linux de 24.5G.

Ahora avanzamos con el formato y el montaje de la estructura para dejar listo el entorno de LFS:

1 Formatear las particiones

1 min

Dale formato FAT32 a la partición EFI y ext4 a la raíz del `loop` device:

```
sudo mkfs.fat -F32 /dev/loop0p1
sudo mkfs.ext4 /dev/loop0p2
```

Verificación: Confirmá que ambos comandos terminen sin errores. (Si te pide sobrescribir firmas previas, respondé `y`).

2 Definir variable y montar directorios

1 min

Configurá la variable `$LFS`, montá la partición raíz y luego integrá la partición EFI en su lugar:

```
export LFS=/mnt/lfs
sudo mkdir -pv $LFS
sudo mount /dev/loop0p2 $LFS
sudo mkdir -pv $LFS/boot/efi
sudo mount /dev/loop0p1 $LFS/boot/efi
```

Verificación: Ejecutá `df -h | grep loop0` para chequear que ambas particiones estén montadas correctamente.

3 Crear la carpeta de sources

30 seg

Creá el directorio para los paquetes y aplícale los permisos que exige el libro:

```
sudo mkdir -v $LFS/sources
sudo chmod -v a+wt $LFS/sources
```

Verificación: Comprobá que la carpeta exista dentro de `$LFS/sources`.

Response: El kernel está reteniendo las particiones del loop device o detectándolas como ocupadas. Para solucionarlo rápido y limpiar el estado actual, vamos a desmontar, desconectar el loop y volver a cargarlo limpiamente con escaneo de particiones.

1 Desconectar y limpiar el loop device

1 min

Desmonta cualquier rastro anterior y libera el dispositivo loop:

```
sudo umount -R /mnt/lfs 2>/dev/null
sudo losetup -d /dev/loop0
```

Verificación: Ejecutá `losetup -a` y asegurate de que ya no aparezca `/dev/loop0`.

2 Reconectar la imagen con particionado automático

1 min

Volvé a asociar la imagen al loop device buscando un espacio libre y forzando la detección de tablas de partición:

```
sudo losetup -fP /lfs.img
sudo losetup -a
```

Verificación: Comprá que te devuelva el loop asignado (ej. `/dev/loop0`) y que el kernel reconozca las subparticiones (`loop0p1` y `loop0p2`).

3 Formatear y montar correctamente

1 min

Ahora aplicá el formato y el montaje secuencial sin errores:

```
sudo mkfs.fat -F32 /dev/loop0p1
sudo mkfs.ext4 /dev/loop0p2
export LFS=/mnt/lfs
sudo mkdir -pv $LFS
sudo mount /dev/loop0p2 $LFS
sudo mkdir -pv $LFS/boot/efi
sudo mount /dev/loop0p1 $LFS/boot/efi
```

Verificación: Ejecutá `df -h | grep loop0` para confirmar que ambos puntos quedaron montados de forma limpia.

User prompt: fuckk

Response: Tranquilo, eso pasó porque el dispositivo `/dev/loop0` ya se había desconectado solo o liberado al fallar el comando anterior. Significa que ya no hay nada colgado ahí.

Arrancamos de nuevo desde cero con el archivo `lfs.img` para dejarlo limpio:

1 Asociar el loop device limpio

1 min

Asociá la imagen al siguiente loop disponible y forzá la lectura de particiones:

```
sudo losetup -fP /lfs.img
```

Verificación: Ejecutá `losetup -a` para confirmar qué número de loop te asignó (digamos que es `/dev/loopX`).

2 Formatear el ESP y la raíz

1 min

Formateá las particiones detectadas (reemplazá `loopX` por tu número real, ej: `loop0`):

```
sudo mkfs.fat -F32 /dev/loopXp1
sudo mkfs.ext4 /dev/loopXp2
```

Verificación: Comprabá que terminen sin errores de recursos ocupados.

3 Montar todo correctamente

1 min

Configurá la variable de entorno y montá las rutas:

```
export LFS=/mnt/lfs
sudo mkdir -pv $LFS
sudo mount /dev/loopXp2 $LFS
sudo mkdir -pv $LFS/boot/efi
sudo mount /dev/loopXp1 $LFS/boot/efi
```

Verificación: Ejecutá `df -h | grep loop` para verificar que ambos puntos queden montados.

4 Crear sources

30 seg

```
sudo mkdir -v $LFS/sources
sudo chmod -v a+wt $LFS/sources
```

User prompt: puta madre

Response: ਕੋਈ ਗੱਲ ਨਹੀਂ, ਗਲਤੀ ਇਹ ਹੋਈ ਕਿ ਤੁਸੀਂ ਹੁਕਮ ਵਿੱਚ `loopXp1` ਅਤੇ `loopXp2` ਲਿਖ ਦਿੱਤਾ, ਜਦੋਂ ਕਿ ਉੱਥੇ ਅਸਲ ਨੰਬਰ (ਜਿਵੇਂ ਕਿ `loop0p1` ਅਤੇ `loop0p2`) ਪਾਉਣਾ ਸੀ। ਆਓ ਇਸਨੂੰ ਬਿਲਕੁਲ ਸਹੀ ਤਰੀਕੇ ਨਾਲ ਦੁਬਾਰਾ ਕਰੀਏ।

1 ਲੂਪ ਡਿਵਾਈਸ ਨੂੰ ਚੈਕ ਕਰੋ ਅਤੇ ਨੰਬਰ ਦੇਖੋ

1 ਮਿੰਟ

ਦੇਖੋ ਕਿ ਸਿਸਟਮ ਨੇ ਕਿਹੜਾ ਲੂਪ ਨੰਬਰ ਦਿੱਤਾ ਹੈ:

```
losetup -a
```

ਜਾਂਚ: ਮੰਨ ਲਓ ਕਿ ਇਹ `/dev/loop0` ਹੈ।

2 ਫਾਰਮੈਟ ਕਰੋ (ਸਹੀ ਨਾਮ ਨਾਲ)

1 ਮਿੰਟ

ਹੁਣ ਸਹੀ ਨਾਮ (`loop0p1` ਅਤੇ `loop0p2`) ਵਰਤ ਕੇ ਫਾਰਮੈਟ ਕਰੋ:

```
sudo mkfs.fat -F32 /dev/loop0p1
sudo mkfs.ext4 /dev/loop0p2
```

3 ਮਾਊਂਟ ਕਰੋ

1 ਮਿੰਟ

ਆਪਣੀ ਵੇਰੀਏਬਲ ਸੈੱਟ ਕਰਕੇ ਮਾਊਂਟ ਕਰੋ:

```
export LFS=/mnt/lfs
sudo mkdir -pv $LFS
sudo mount /dev/loop0p2 $LFS
sudo mkdir -pv $LFS/boot/efi
sudo mount /dev/loop0p1 $LFS/boot/efi
```

User prompt: nope

Response: Ahí está el problema en la captura: ejecutaste `losetup -fP /lfs.img` varias veces y se abrieron tres loop devices distintos (`loop0`, `loop1` y `loop2`) apuntando al mismo archivo, lo que hizo que el kernel se haga un lío con las particiones.

Limpiemos todo de raíz y hagámoslo prolijo en un solo intento:

1 Desconectar todos los loop devices basura

1 min

Liberá cualquier dispositivo loop que esté usando la imagen:

```
sudo umount -R /mnt/lfs 2>/dev/null
sudo losetup -d /dev/loop0 2>/dev/null
sudo losetup -d /dev/loop1 2>/dev/null
sudo losetup -d /dev/loop2 2>/dev/null
```

Verificación: Ejecutá `losetup -a` y asegurate de que no quede ninguno activo.

2 Asociar un único loop device con particiones

1 min

Conectá la imagen limpiamente asignando uno solo y escaneando sus particiones:

```
sudo losetup -fP /lfs.img
losetup -a
```

Verificación: Fijate qué número te devolvió (por ejemplo, supongamos que te asignó `loop3`). De ahora en adelante usá ese número exacto.

3 Formatear y montar con el número correcto

1 min

Aplicá el formato y el montaje usando el loop que te dio (reemplazá `loop3` por el tuyo si fue otro):

```
sudo mkfs.fat -F32 /dev/loop3p1
sudo mkfs.ext4 /dev/loop3p2
export LFS=/mnt/lfs
sudo mkdir -pv $LFS
sudo mount /dev/loop3p2 $LFS
sudo mkdir -pv $LFS/boot/efi
sudo mount /dev/loop3p1 $LFS/boot/efi
```

Verificación: Ejecutá `df -h /mnt/lfs` para confirmar que quedó montado correctamente.

Response: Claro, ahí está el detalle clave en la captura: el comando `losetup -a` te devolvió `/dev/loop0` (no `/dev/loop3`), pero en los comandos siguientes le pusiste `loop3p1` y `loop3p2` por error. Como el loop 3 no existía, el sistema tiró que no encontraba los archivos.

Hagámoslo bien directo usando el loop que realmente asignó (`loop0`):

1 Verificar el loop activo

30 seg

```
losetup -a
```

Verificación: Confirmá que el loop asignado sea `/dev/loop0`.

2 Formatear usando loop0p1 y loop0p2

1 min

```
sudo mkfs.fat -F32 /dev/loop0p1
sudo mkfs.ext4 /dev/loop0p2
```

Verificación: Ambos comandos deben terminar sin errores.

3 Montar la estructura LFS

1 min

```
export LFS=/mnt/lfs
sudo mkdir -pv $LFS
sudo mount /dev/loop0p2 $LFS
sudo mkdir -pv $LFS/boot/efi
sudo mount /dev/loop0p1 $LFS/boot/efi
```

Verificación: Ejecutá `df -h /mnt/lfs` para confirmar que quedó montado correctamente.

4 Crear la carpeta sources

30 seg

```
sudo mkdir -v $LFS/sources
sudo chmod -v a+wt $LFS/sources
```

Response: ¡Excelente! Ahí se ve que formateó el `loop0p2` en `ext4` sin ningún problema.

Ahora solo te falta terminar de montar las particiones y crear la carpeta de fuentes para dejar todo listo:

1 Montar las particiones en /mnt/lfs

1 min

Ejecutá los comandos de montaje y la carpeta de sources:

```
export LFS=/mnt/lfs
sudo mkdir -pv $LFS
sudo mount /dev/loop0p2 $LFS
```

```
sudo mkdir -pv $LFS/boot/efi
sudo mount /dev/loop0p1 $LFS/boot/efi
```

Verificación: Ejecutá `df -h /mnt/lfs` para confirmar que esté todo montado correctamente.

2 Preparar la carpeta de sources

30 seg

```
sudo mkdir -v $LFS/sources
sudo chmod -v a+wt $LFS/sources
```

Verificación: Confirmá que la carpeta exista y tenga los permisos correctos.

Response: ¡Hermoso! Ahí se ve en la captura cómo montó el sistema raíz, creó el directorio de boot/efi, montó la partición ESP y configuró perfectamente la carpeta `$LFS/sources` con los permisos `1777 (rwxrwxrwt)` tal cual exige el libro de LFS.

Con el entorno ya montado y preparado, el siguiente paso según el libro es la **descarga de todos los paquetes fuente** necesarios en `$LFS/sources`.

¿Querés que pasemos a armar el archivo `wget-list` y descargarnos los paquetes, o preferís verificar algo más del montaje primero?

User prompt: sisi

Response: Perfecto, avancemos entonces con la descarga de los paquetes en la carpeta de fuentes.

Para hacerlo de forma limpia, necesitamos el archivo `wget-list` con las URLs de los paquetes que especifica la versión del libro de LFS que estás siguiendo.

¿Ya tenés a mano el archivo `wget-list` o querés que armemos el comando para descargarlo directamente desde el sitio oficial de LFS?

User prompt: Asociá un loop device Conectá la imagen como si fuera un disco físico: 'sudo losetup -fP nethos-lfs.img' y después 'losetup -a' para ver qué '/dev/loopN' te asignó (anotá ese número, lo vamos a usar todo el tiempo). 3Particioná con GPT + ESP Particioná en modo GPT (obligatorio para UEFI, igual que necesita la G13 real): 'sudo cfdisk /dev/loopN'. Creá dos particiones: una de ~512MB tipo 'EFI System' (para el ESP) y el resto tipo 'Linux filesystem' (para la raíz). Guardá los cambios al salir. 4Formatá las particiones Formatá cada partición: la ESP como FAT32 ('sudo mkfs.fat -F32 /dev/loopNp1') y la raíz como ext4 ('sudo mkfs.ext4 /dev/loopNp2'). Estos son los mismos comandos que vas a repetir después en la G13 real. 5Montá \$LFS Montá la partición raíz como '\$LFS' (la variable que usa todo el libro): 'export LFS=/mnt/lfs', 'sudo mkdir -pv \$LFS', 'sudo mount /dev/loopNp2 \$LFS'. Después creá y montá el ESP dentro: 'sudo mkdir -pv \$LFS/boot/efi' y 'sudo mount /dev/loopNp1 \$LFS/boot/efi'. 6Prepará la carpeta de sources Creá la carpeta de sources con permisos correctos según pide el libro: 'sudo mkdir -v \$LFS/sources', 'sudo chmod -v a+wt \$LFS/sources'. Ahí van a caer los ~90 tarballs que vamos a descargar en el próximo paso.. ya hicimos todo esto?

Response: ¡Exacto! Todo ese bloque ya lo completamos con éxito en los pasos anteriores:

1. **Loop device conectado:** Asociamos el archivo de imagen a `/dev/loop0` y el kernel reconoció las particiones sin problema.

2. **Particionado GPT + ESP:** Creamos la tabla GPT con `cfdisk`, la partición ESP de 512MB y la raíz con el espacio restante.
3. **Formateo:** Aplicamos formato FAT32 a `loop0p1` y `ext4` a `loop0p2`.
4. **Montaje de \$LFS:** Configuramos la variable `$LFS=/mnt/lfs`, montamos la partición raíz, creamos `$LFS/boot/efi` y montamos el ESP dentro.
5. **Carpeta de sources:** Creada en `$LFS/sources` con los permisos exactos `1777 (a+wt)` que pide el libro de LFS.

Ya tenemos el terreno 100% listo para pasar a la descarga de los paquetes en `$LFS/sources`. ¿Querés que avancemos con eso?
